FIELD BRIEF

Anti-Dark-Code

A staged workflow that turns an unfamiliar or messy codebase into a mapped, documented, evidence-backed one, so an AI (or a person) works from proof instead of guesswork.

EVIDENCE OVER GUESSING · ONE SLICE AT A TIME · APPROVAL BEFORE RISKY EDITS · DETERMINISTIC FIRST · ONE CORE, ANY HOST · OPTIONAL AI SWARM

Plain-language brief. Covers what the skill does, every pass, the evidence rules, the deterministic verification layer, how a repo installs and calibrates its own copy, and how the agent swarm runs when the harness supports it.

What problem it solves

Every long-lived codebase collects **dark code**: parts that look understandable but hide how the system actually behaves. Unclear ownership, silent side effects, a script that quietly touches production, prose that promises a feature the code never implements. Dark code is dangerous because it makes people confident about things they cannot actually prove.

Anti-Dark-Code is the antidote. It gives an AI a fixed set of numbered passes to run, one at a time, each producing durable records the repo keeps. The rule underneath all of it is short: do not guess. Mark every claim as proven, likely, or unknown, and never let a likely claim get written down as a proven one.

> **The one idea to remember**
> Facts come from evidence in the repo, not from the model's memory or a confident guess. If the code cannot prove it, the skill will not claim it. That single habit is what keeps the whole workflow honest.

The evidence language

Every claim the skill records carries one of three labels. This is the vocabulary the whole workflow shares.

`verified` Direct repo evidence supports the claim. It cites the exact file, line, command, or document that proves it.

`inferred` The evidence is consistent with the claim but does not prove it. The skill names the gap that stops it from being verified.

`unknown` Evidence is missing, contradictory, or lives outside the repo (a vendor dashboard, a sibling repo, live telemetry). Recorded, not guessed at.

When a check would force a claim down to `inferred`, the skill writes `inferred`. It downgrades rather than rounds up. Anything it cannot resolve becomes a written `unknown` with an owner, a risk level, and the next best check, so nothing quietly disappears.

The rules that never bend

Work one slice at a time

Run one numbered pass on one bounded slice, finish it, record it, then move on. No interleaving passes. Small, reversible steps beat one sweeping change.

Stop at approval gates

Auth, billing, deletion, secrets, migrations that rewrite saved data, hash or receipt truth: document the finding first, propose the smallest safe edit, and stop for a human before touching it.

Two more hold across every pass. **Preserve scope honesty:** one clean file does not stand in for a whole mixed system, and out-of-repo control planes are named, not folded into local coverage. **Protect the record:** comments that explain a trust boundary, an ordering rule, or a recovery path are treated as load-bearing and are not silently removed.

The passes

The skill is a menu of numbered passes grouped into families. A run loads only the one pass it is doing, does that work, reports back, and picks the next. Most passes are read-only or docs-only. Only the remediation pass changes application code, and only after the map, the unknowns, and the approval gates are in place.

ORIENT

00 **Preflight**

Look before committing. Sniff what kind of repo this is, check whether earlier maps have gone stale, and pick where to start.

01 **Steering files**

Read and set the operating rules: the AGENTS.md style guidance, approval gates, and sensitive-data rules that govern the repo.

MAP

02 **Architecture map**

Draw the real runtime picture: what runs where, where data enters, and where trust changes hands.

05 **Coverage and slicing**

When the repo is too big for one honest pass, cut it into risk-ranked slices and keep an honest ledger of what is covered, blocked, or still dark.

HARDEN

03 **Critical-path comments**

Add why-focused comments on the riskiest code, without changing behavior.

04 **Logging and telemetry audit**

Inspect logs, analytics, and error paths for leaks and over-collection.

06 **Writing hygiene**

Clean vague, inflated, or sloppy text out of the comments and docs the earlier passes wrote.

CHALLENGE

07 **Adversarial review**

Try to break the current picture. Hunt blind spots, weak trust boundaries, hidden control planes, and coverage that claims more than it proved.

08 **Scenario stress-test**

Run realistic failure and abuse scenarios against the map to see whether it holds.

CLEAN UP AND KEEP CLEAN

09 **Artifact garbage collection**

Separate the claim from the file, record how to regenerate it, then tier the clutter (logs, snapshots, scratch scripts, stale baselines) and clean it up safely.

10 **Maintenance harness**

Install guardrails: PR templates, drift checks, and review gates that keep the repo honest over time.

12 **Transcreation boundary**

Keep language, locale, and copy from becoming hidden runtime truth. Rendered text is a view, not the source of facts.

FIX

11 **Remediation loop**

Turn findings into small, safe, approval-gated fix batches, each with a verification step. This is the only pass that edits application code.

CALIBRATE AND VERIFY

13 **Calibrated local mode**

Install the skill into the repo itself, with a repo-owned calibration layer (invariants, system map, exact gates, coverage and findings ledgers) that every later pass reads first. A known repo stops pretending to be unfamiliar every time a context window resets.

14 **Deterministic verification planner**

Profile the repo with a bounded script, evaluate all twenty verification capabilities (mutation, fuzzing, goldens, replay, metamorphic properties, fault injection, and more), select the repo-fit subset, and store the exact reviewed commands the computer will run with real exit codes.

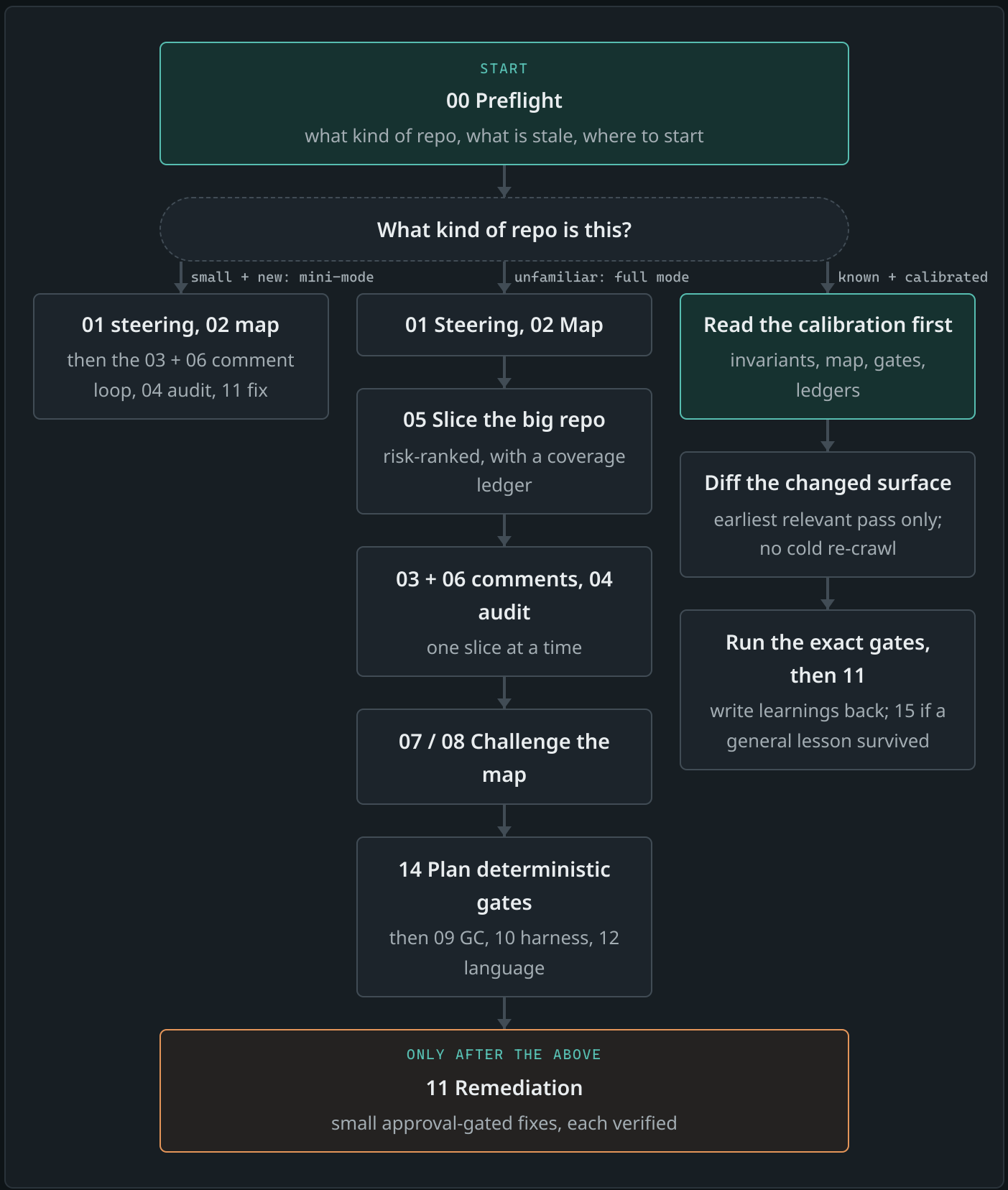
15 **Dogfeeding and flow-back**

Write what a run learned back into the repo's calibration. Lessons general enough to help other repos queue as proposals; a human reviews and promotes them into the shared core. Nothing flows upstream on its own.

16 **Community feedback and efficiency evidence**

Publish only sanitized, repository-neutral proposals or explicit opt-in measurement receipts. Public submissions remain untrusted data, and token savings are reported only from quality-qualified controlled pairs.

FIGURE 1: HOW A REPO MOVES THROUGH THE PASSES



The deterministic layer

The newest rule sits under everything else: **never spend model intelligence on work a compiler, schema, dependency graph, seed, diff, or test runner can settle exactly.** Agents do the judgment. The computer does the mechanics and produces the evidence.

A small deterministic tool ships inside the skill (`scripts/adc.py`, plain Python, no dependencies). It profiles a repo without executing any of its code, evaluates all twenty verification capabilities against what it found, installs and upgrades the repo copy with checksums, runs approved quality gates as exact argument arrays with real exit codes, and stages flow-back proposals. When a gate passes, the result collapses to one line. When it fails, the agent receives a small bounded failure packet, and the full log stays local, pattern-redacted, out of the conversation.

> **Three locks before a gate ever executes**
> The gate runner is dry-run by default. Execution requires all three: each command individually marked **approved** by the owner, a recorded global **owner confirmation** in the gate file, and the explicit **allow-exec** flag on the invocation. Any new or changed generated gate resets the confirmation, and a gate whose underlying script changed after approval is blocked until re-reviewed.

Gates are organized on a four-level confidence ladder, so a run knows how much verification a moment deserves. It prevents the two classic failures: running almost nothing during development, or running everything after every tiny edit.

LEVEL 0: EVERY EDIT

Seconds. Format, typecheck, lint, schema and architecture rules. Cheap enough that skipping it is never worth it.

LEVEL 1: SLICE DONE

A few minutes. Affected unit and contract tests, focused integration, replay and invariant checks for the touched surface.

LEVEL 2: PRE-MERGE

Selective depth. Property and metamorphic checks, short fuzz runs, changed-module mutation, perf smoke, targeted fault injection.

LEVEL 3: SCHEDULED

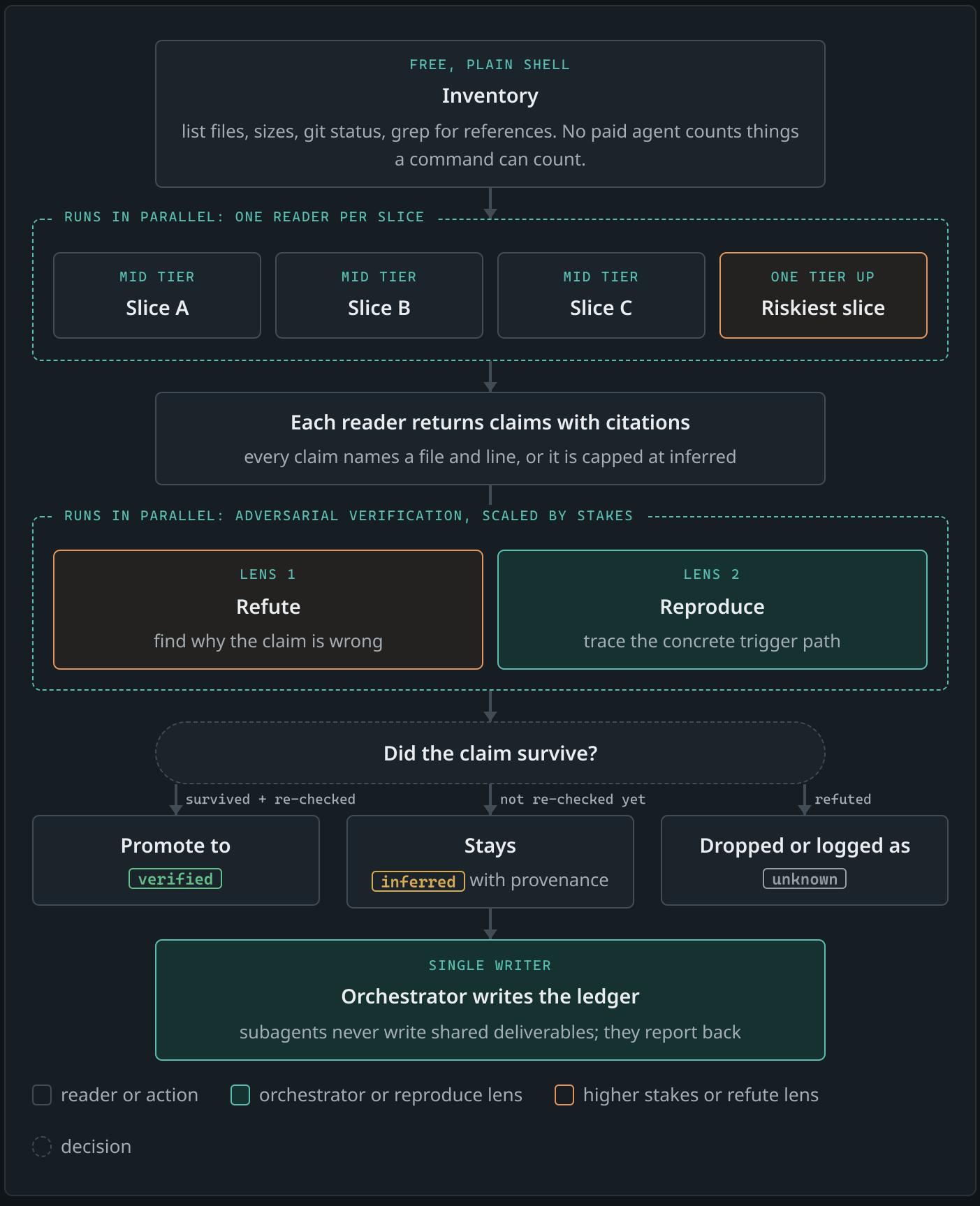
The heavy battery. Full suites, long fuzz and soak campaigns, migration and platform matrices, statistical canaries for emergent behavior.

How the AI swarm works

On a harness that can spawn subagents, the skill switches on an **orchestration mode**. This changes only *how* the work runs, never the pass order, the approval gates, or the evidence rules. The most important boundary: the fan-out lives **inside a single pass**. Many readers, many slices, many verification angles, but still one pass at a time. Passes never run in parallel with each other.

A pass that fans out follows the same shape every time. Free shell work does the counting. Mid-tier readers do the reading. Independent verifiers try to break each claim. One orchestrator writes the record. A subagent's claim never enters the record as proven on its own word.

FIGURE 2: THE SWARM INSIDE ONE PASS



> **The promotion rule, in one line**
> A subagent's claim enters the record as `inferred`. It becomes `verified` only when the orchestrator personally re-opens the citation, or two independent readers with different angles agree, or a command settles it. Effort is spent re-checking the highest-risk claims first.

Choosing the model, and adjusting mid-run

Model choice is spoken in plain tiers, not product names, so it travels to any harness: cheap-and-wide for the reading, a step up for the riskiest slice, top tier and few calls for synthesis. There are two dials, not one. **Tier** raises the capability ceiling. **Effort** buys more thinking on the same ceiling.

Verification produces a free signal for tuning them. At each checkpoint the orchestrator counts how many claims per family survived, then adjusts the next batch:

- Claims refuted as **misunderstandings** of how parts interact mean the reader hit a ceiling. Raise the tier, or stop that family and bank what survived.
- Claims that failed from **shallow coverage** (stopped early, traced one path of four) mean not enough thinking. Raise the effort.
- **No failures** means the recipe works. Change nothing. A winning recipe is not an invitation to tune.

> **Budget and recovery are first-class**
> The orchestrator quotes the fan-out shape (how many agents, at what tier) before launching, especially after any usage limit. Each unit of paid work is its own call with a stable prompt, so if a run dies partway it resumes from cache: completed work replays for free, and only the unfinished calls run again. Progress is read straight from the run journal with shell tools, never by paying an agent to summarize it.

Pass 09 up close: cleaning up without losing anything

Agent-driven work leaves debris the same way old systems leave dark code: gigabytes of logs, hundreds of snapshots, one-off scripts nobody can explain. Pass 09 handles it with one principle first. The lasting value of a generated file is the **claim it supports** plus the **recipe that regenerates it**. Record both, then the file itself drops into one of four tiers and gets handled by the tier's rule, not by a hunch.

PROTECTED

Never touched. Anything git-tracked, or named by a steering file (docs, review artifacts, assets, history). A broad "clean it all up" does not override this.

REGENERABLE, CHEAP

Record the recipe, then remove. One command or seed rebuilds it fast. Archive as a short-retention fallback, spot-check that it rebuilds, then delete the originals.

REGENERABLE, EXPENSIVE

Archive first, delete later, never the reverse. Checksum, compress, verify by re-hashing, add parity data when the archive is the only copy, and ask before removing originals.

UNKNOWN PROVENANCE

Hard stop. Nobody can explain it. Archive it untouched or leave it in place, record an unknown, and ask. Absence of references proves it is unused by current code, not that it is safe to delete.

Deletion is always the second step, never the first. Everything moves through a reversible quarantine, gets verified against a checksum manifest, and only then, with per-group approval, gets removed. A tracked ledger records where each thing went and how to get it back, so a future session can answer that from the record alone.

One core, many hosts, one truth per repo

The skill used to ship as parallel Claude and Codex variants, and they drifted. It is now **one model-neutral core**. Host differences (how Claude Code, Codex, or Gemini CLI discovers a skill, which tools it prefers) live in small addendum files inside the core, never as separate editable trees.

- **The shared core** lives in one version-controlled directory. Each host's skill folder is just a pointer to it (a symlink or a thin adapter), so an upgrade happens exactly once. A marker file identifies it as a clean universal source; the installer refuses to install from anything else.
- **The repo copy** is real, not a pointer: installing places a managed, checksummed copy inside the repository so it travels with the code to any machine or CI runner. The managed core is read-only in the repo; local edits surface as conflicts on the next upgrade instead of silently forking.
- **The calibration overlay** is the repo's own memory: invariants, system map, exact gates, coverage and findings ledgers. It is owned by the repo, survives every core upgrade, and is what makes pass 13's calibrated mode possible.

- **The upgrade preview** is read-only: before an install changes anything, it reports which deterministic gates would change and whether owner approval would reset. A reviewed repo-owned gate survives a bounded refresh while its exact source binding still verifies.
- **The repo binding** stamps the calibration with a hashed identity of its repository (no raw remotes, no personal paths). Calibration copied from another project will not silently run here: gates from unbound or foreign calibration are refused until a human explicitly accepts or rebinds them, and accepting resets every gate back to unapproved.

> **The trust barrier**
> Knowledge flows one way by default: the clean core flows down into repos; repo knowledge stays in its repo. The only path upstream is a sanitized, content-hashed **proposal** staged into the shared core's review inbox, which a human reads, generalizes, and promotes. A compromised or simply overfitted repository cannot rewrite the shared skill and poison every future project.

Token efficiency, without make-believe math

Exact savings before this measurement protocol existed are `unmeasured`. The skill makes no automatic network calls and sends no telemetry. If a host reports the tokens used during one assisted run, that is **usage**, not proof of savings.

A tokens-saved claim requires a quality-qualified controlled pair: the same provider, model, counter method and adapter, equivalent task and repository state, equivalent tool access, the same acceptance contract with both outcomes passing, and one run with the skill compared with one baseline run without it. Tokenizer or context-compaction estimates are useful engineering proxies, but remain separate from provider-reported token counts. Public community receipts are opt-in and self-reported, never provider-attested.

> **Public measurement status**
> This static brief does not invent a historical total. The live public brief displays validated, self-reported controlled-pair evidence when a public summary exists.

What you end up with

Every pass reports back the same way: which pass ran on which slice, what was created or updated, what unknowns were recorded, which risks moved, which approval gates were crossed or are still pending, and what to run next. The durable outputs live in the repo:

- A **system map** of what runs where and where trust changes hands.
- A **coverage ledger** that says honestly what is covered, blocked, approval-gated, or still dark.

- **Unknowns files** that preserve every real gap instead of smoothing it over.
- **Why-focused comments** on the critical paths, and a **remediation backlog** of small, safe fixes.
- A **verification plan** covering all twenty capabilities, and **exact reviewed gates** the computer runs with real exit codes.
- A **calibration layer** bound to the repo, so the next session starts from recorded truth instead of a cold crawl.
- A **maintenance harness** so the next change stays honest.

When to reach for it, and when not

Reach for it when

- You inherited a repo nobody fully understands.
- You need to know what runs where and where the trust boundaries are.
- You want risky work held behind evidence and approval, not vibes.
- Generated clutter is piling up and needs safe cleanup.
- You want future changes to stay safe, not just this one to work.

Skip it when

- The task is a quick, well-understood change in code you already know.
- A regex or a validator can enforce the thing mechanically. Automate it instead.
- You only need a one-line answer, not a mapped and recorded pass.

Anti-Dark-Code field brief, updated 2026-08-20 for the unified core (2026.08.20-unified.6): one model-neutral skill with thin host adapters for Claude, Codex, and Gemini harnesses. Assurance contracts, bounded public proposal intake, and opt-in efficiency receipts are the newest additions.